HTML Engineering Notes — Audit & Learning Roadmap

Core2Web Classroom Lecture | 14 October 2026
Language: Marathi + English | **Target:** Strong fundamentals through 3+ years engineer-level understanding

How to use this document

- **Original classroom notes** are reproduced in the next section without intentionally rewriting their wording.
- **Audit and technical addendum** distinguishes corrections and expansions from the original material.
- If an original classroom point is simplified, the addendum gives the more precise engineering model. When exact classroom wording is uncertain, do not treat the reconstructed wording as a verbatim quote from the notebook.
- Code examples are experiments: run them in a local HTML file and inspect both **View Source** and **DevTools → Elements**.

Audit summary: what is strong and what needs care

Strongly covered in the source

Internet vs WWW; UNIX and ARPANET context; CERN and Tim Berners-Lee; HTML history; HTML/CSS/JavaScript roles; tags, elements and void elements; document skeleton; metadata, UTF-8 and viewport; SEO basics; DOM tree; browser rendering; Chrome, Blink and V8; source versus DOM; implicit `<tbody>`; missing body/head/title/DOCTYPE; C++-class analogy; `innerHTML` versus `textContent`; XSS; `alt`; clickable `<div>` versus `<button>`; form button defaults; deferred scripts; nesting and debugging questions.

Technical precision corrections to remember

1. **HTML5 and versions:** 2014 was a major HTML5 Recommendation milestone. Modern HTML continues to evolve as the WHATWG HTML Living Standard; HTML is not frozen at “HTML5.”
2. **Parser recovery:** Browsers use defined HTML parsing algorithms and often repair malformed markup. This does not mean every invalid document is repaired in the same way, nor that the source itself is rewritten.
3. **DOCTYPE:** `<!doctype html>` primarily selects standards mode in modern browsers. Omitting it commonly triggers quirks mode; it does not simply make the page fail to parse.
4. **DOM implementation:** DOM interfaces are web-platform APIs. Browser internals may use C++, Rust, or other implementation techniques. An HTML tag is not literally a C++ class.
5. **Rendering pipeline:** Parsing, style calculation, layout, paint, rasterization and compositing are useful conceptual stages, but browsers may pipeline, parallelize, cache or skip work depending on changes.

6. **Encoding:** UTF-8 is a character encoding (Unicode code points encoded as one to four bytes). Correct interpretation also depends on how the browser learns the document encoding; the `<meta charset="utf-8">` declaration should appear early in the document head.
7. **Security:** `textContent` is generally appropriate for inserting untrusted text. `innerHTML` parses markup and can introduce XSS when used with unsafe untrusted content. Context-aware output encoding, sanitization when HTML is intentionally allowed, and a defense-in-depth approach are still required.
8. **HTML vs XHTML:** HTML permits `<br>` and also accepts `<br/>` syntax. In XML/XHTML serialization, well-formed XML syntax rules apply. `<br>` is a void element and must not have a closing tag in HTML.
9. **Form controls:** A `<button>` associated with a form defaults to submit behavior unless its type says otherwise. Use `type="button"` for non-submit actions.
10. **SEO and accessibility:** Semantic HTML helps machines and assistive technology understand content, but no single tag guarantees rankings or accessibility by itself.

Audit gaps addressed in the addendum

- Semantic structure and landmark elements
- Attributes, boolean attributes, global attributes and `data-*`
- Forms: labels, name/value, GET/POST, constraint validation and submit behavior
- Accessibility tree and keyboard interaction
- Script loading: parser-blocking classic scripts, `defer`, `async`, and modules
- CSSOM, render-blocking resources, layout/reflow, paint and compositing
- Event loop, long tasks and main-thread responsiveness
- Resource loading, caching, responsive images and lazy loading
- Security: XSS, URL schemes, safe DOM APIs and CSP as defense in depth
- Performance and production debugging workflow
- More robust practical experiments and interview answers

Original Classroom Notes — Preserved Source

The following section reproduces the uploaded text as the source material. Corrections and additions are separated into the addendum after it.

मी तुझ्या पाठवलेल्या सर्व 8 images काळजीपूर्वक पाहिल्या आहेत. काही ठिकाणी handwriting अस्पष्ट आहे, त्यामुळे जिथे एखादा शब्द निश्चितपणे वाचता येत नाही तिथे मी अंदाज लावून तो अचूक असल्याचा दावा करणार नाही. मात्र, वाचता येणारे classroom points, code, diagrams आणि technical terms एकत्र करून खाली व्यवस्थित notes तयार केल्या आहेत.

या notes मध्ये तीन गोष्टी वेगळ्या ठेवल्या आहेत:

- **CLASS NOTES:** तुझ्या वहीत दिसणारे मुद्दे.

- **TECHNICAL EXPLANATION:** त्या मुद्द्यांचे सविस्तर स्पष्टीकरण.
- **ADVANCED INSIGHT:** 3+ years experience असलेल्या engineer ला समजणे आवश्यक असलेले संबंधित concepts.

HTML — Introduction to HTML

Core2Web Classroom Lecture — 14 October 2026

Notebook Notes | Marathi + English | Engineering-Level Understanding

---

1. History of the Web and HTML

तुझ्या वहीमध्ये Internet, UNIX, ARPANET, HTML, CERN, Tim Berners-Lee आणि HTML versions यांच्याशी संबंधित मुद्दे दिसतात.

हे concepts समजण्यासाठी आधी Web चा इतिहास समजून घेऊ.

1.1 UNIX — 1969

Definition: UNIX is a multiuser, multitasking operating system originally developed at Bell Labs.

मराठीत:

UNIX ही एक Operating System आहे. तिच्यावर अनेक users एकाच वेळी काम करू शकतात आणि अनेक processes execute होऊ शकतात.

UNIX चा development 1969 मध्ये Bell Labs येथे सुरू झाला.

Real-life example:

समजा, एका college च्या server वर अनेक students एकाच वेळी login करून वेगवेगळी कामे करतात. Operating System या सर्व processes आणि system resources manage करते.

UNIX आणि त्यावर आधारित operating systems यांचा networking आणि server computing च्या विकासात मोठा वाटा आहे.

Important distinction:

- UNIX ही Operating System आहे.
- Internet हे interconnected networks चे जागतिक जाळे आहे.
- WWW ही Internet वर चालणारी एक service आहे.
- HTML ही Web pages ची structure define करणारी markup language आहे.

या चार गोष्टी एकसारख्या नाहीत.

1.2 ARPANET

Full form: Advanced Research Projects Agency Network.

ARPANET हे आधुनिक Internet च्या विकासातील महत्त्वाचे precursor होते.

1969 मध्ये ARPANET वर सुरुवातीचे network connections स्थापित झाले.

मराठीत:

पूर्वी वेगवेगळ्या ठिकाणी असलेल्या computers मधील communication सोपे नव्हते. ARPANET सारख्या networks मुळे दूरवरच्या computers मध्ये data exchange करण्याचा पाया तयार झाला.

याच networking research मधून आधुनिक Internet च्या विकासाला चालना मिळाली.

** लक्षात ठेव:**

ARPANET आणि आजचे Internet एकच गोष्ट नाहीत. ARPANET हे Internet च्या इतिहासातील एक महत्त्वाचे सुरुवातीचे network होते.

1.3 WWW — World Wide Web

Definition: The World Wide Web is a system of interlinked resources that can be accessed over the Internet using technologies such as URLs, HTTP and HTML.

मराठीत:

World Wide Web म्हणजे एकमेकांशी links ने जोडलेली Web pages, documents, images, videos आणि इतर resources यांची व्यवस्था.

उदाहरणार्थ, तू browser मध्ये एखादी website उघडतोस.

त्या website वर:

- Text असतो.
- Images असतात.
- Videos असतात.
- इतर pages कडे जाणाऱ्या links असतात.

हे resources Web technologies च्या मदतीने उपलब्ध होतात.

महत्त्वाचा फरक:

Internet म्हणजे network infrastructure, तर WWW ही त्या infrastructure वर चालणारी service आहे.

उदाहरण:

Internet ला रस्त्यांच्या जाळ्यासारखे समज.

WWW म्हणजे त्या रस्त्यांचा वापर करून वेगवेगळ्या ठिकाणी उपलब्ध होणाऱ्या services आणि resources पैकी एक व्यवस्था.

Internet वर Email, online gaming आणि इतर services देखील चालतात; त्या सर्व WWW चाच भाग आहेत असे नाही.

---

2. Tim Berners-Lee आणि HTML चा इतिहास

तुझ्या वहीत खालील नावे स्पष्टपणे दिसतात:

- CERN
- Tim Berners-Lee
- HTML
- HTTP
- HTML versions
- Metadata
- Browser

हे सगळे एकमेकांशी संबंधित आहेत.

2.1 CERN

Full form: European Organization for Nuclear Research.

CERN ही particle physics research साठी प्रसिद्ध असलेली आंतरराष्ट्रीय संशोधन संस्था आहे.

तिथे विविध देशांतील scientists संशोधन करत असत. त्यांना वेगवेगळ्या computers वर उपलब्ध असलेली माहिती परस्परांशी जोडून सहज access करता यावी, अशी गरज होती.

याच प्रकारच्या समस्येच्या पार्श्वभूमीवर World Wide Web चा विकास झाला.

2.2 Tim Berners-Lee

Tim Berners-Lee यांनी 1989 मध्ये CERN येथे World Wide Web ची कल्पना प्रस्तावित केली.

1990 मध्ये Web च्या पहिल्या implementation वर काम झाले. त्यात Web server, browser/editor आणि HTML, HTTP व URL यांसारख्या मूलभूत तंत्रज्ञानांचा समावेश होता.

HTML म्हणजे काय?

Full form: HyperText Markup Language.

Definition:

HTML is a standard markup language used to structure content on the Web.

मराठीत स्पष्टीकरण:

HTML चा वापर Web page वरील content ची structure define करण्यासाठी केला जातो.

उदाहरणार्थ, एखाद्या page वर:

- Main heading कुठे आहे?
- Paragraph कोणता आहे?
- Image कुठे दाखवायची आहे?
- Link कोणता आहे?

- Form कुठे आहे?

या content ची अर्थपूर्ण रचना HTML define करते.

HTML मधील तीन शब्दांचा अर्थ

HyperText:

एका document मधून दुसऱ्या document किंवा resource कडे link द्वारे जाण्याची संकल्पना.

उदाहरण: एखाद्या article मधील link वर click केल्यावर दुसरे Web page उघडते.

Markup:

Content ची रचना आणि त्याचा अर्थ दर्शवण्यासाठी tags वापरणे.

उदाहरण:

```
<h1>Welcome</h1>

<p>This is a paragraph.</p>
```

इथे `<h1>` heading दर्शवते, तर `<p>` paragraph दर्शवते.

Language:

HTML मध्ये predefined syntax आणि rules आहेत. मात्र, HTML ही Java किंवा Python सारखी general-purpose programming language नाही.

Interview Question

Q. Is HTML a programming language?

Answer:

No. HTML is a markup language, not a general-purpose programming language. It describes the structure and semantics of Web content but does not provide general-purpose programming constructs such as loops and functions in the same way as Java or Python.

---

3. HTML Versions आणि Web चा विकास

तुझ्या वहीत खालील नोंदी दिसतात:

- HTML → 1.0 → 1993
- HTML 2.0 → 1995
- HTML 4.0 → 1997
- HTML5 → 2014
- XHTML 1.0 → 2000

यांचा अर्थ समजून घेऊ.

Version	Year	महत्त्व
HTML 1.0	1993 च्या सुरुवातीच्या Web च्या संदर्भात	सुरुवातीच्या Web pages साठी साधी document structure
HTML 2.0	1995	HTML specification चे औपचारिक standardization
HTML 4.0	1997	CSS, scripting आणि अधिक व्यापक document features साठी समर्थन
XHTML 1.0	2000	XML च्या strict syntax rules नुसार HTML 4 चे reformulation
HTML5	2014	WHATWG आणि W3C च्या ऐतिहासिक standardization प्रक्रियेत महत्त्वाचा टप्पा

टीप: HTML च्या इतिहासातील काही वर्षे वेगवेगळ्या specifications आणि त्यांच्या publication milestones नुसार वेगळी दिली जाऊ शकतात. वरील वर्षे महत्त्वाच्या ऐतिहासिक टप्प्यांचा संदर्भ देतात.

HTML5 बद्दल महत्त्वाचे

HTML5 मुळे आधुनिक Web development मध्ये खालील गोष्टींसाठी native support उपलब्ध झाला:

- `<audio>` आणि `<video>`
- Semantic elements, जसे `<header>`, `<nav>`, `<main>` आणि `<footer>`
- `<canvas>` द्वारे graphics rendering
- सुधारित forms आणि input types

आज HTML चे standardization आणि development मुख्यतः WHATWG च्या Living Standard द्वारे पुढे चालते.

म्हणून HTML5 ला फक्त एक नवीन version समजण्याऐवजी आधुनिक HTML च्या विकासातील एक महत्त्वाचा टप्पा म्हणून समजणे योग्य आहे.

---

4. HTML, CSS आणि JavaScript

तुझ्या वहीतील diagram मध्ये HTML, CSS आणि Browser यांचा संबंध दाखवलेला दिसतो.

हा Web development मधील सर्वात महत्त्वाचा fundamental concept आहे.

HTML — Structure

HTML content ची रचना define करते.

उदाहरण:

```
<h1>My Website</h1>

<p>Welcome to my website.</p>
```

इथे:

- `<h1>` हा main heading आहे.
- `<p>` हा paragraph आहे.

CSS — Presentation

CSS page कसा दिसेल हे नियंत्रित करते.

उदाहरण:

```
h1 {
    color: blue;
    font-size: 32px;
}
```

इथे heading चा रंग आणि font size define केला आहे.

JavaScript — Behavior

JavaScript च्या मदतीने Web page मध्ये dynamic behavior तयार करता येतो.

उदाहरण:

```
document.querySelector("h1").textContent = "Hello World";
```

हा code page वरील पहिल्या `<h1>` element चा text बदलतो.

Real-life example

समजा, तू Amazon सारखी E-commerce website बनवत आहेस.

Technology	भूमिका
HTML	Product title, price, image, buttons यांची structure
CSS	Colors, layout, spacing, responsive design
JavaScript	Add to Cart click, dynamic updates, interactions

Advanced insight:

मोठ्या Web applications मध्ये React सारख्या libraries आणि Spring Boot सारख्या backend frameworks चा वापर होऊ शकतो. मात्र, त्यातून तयार होणाऱ्या Web interfaces च्या पायामध्ये HTML, CSS आणि JavaScript या मूलभूत Web technologies असतात.

---

5. HTML Tag vs HTML Element

तुझ्या वहीत हा मुद्दा स्पष्ट दिसतो:

Tag vs Element

तसेच `<br>` साठी void tag किंवा self-closing tag अशी नोंद दिसते.

हा फरक व्यवस्थित समजून घेऊ.

5.1 HTML Tag

Tag म्हणजे HTML markup मधील delimiters आणि tag name वापरून लिहिलेली रचना.

उदाहरण:

```
<p>
```

हा opening tag आहे.

```
</p>
```

हा closing tag आहे.

5.2 HTML Element

Element म्हणजे HTML document मधील संपूर्ण construct.

उदाहरण:

```
<p>Hello World</p>
```

यात:

- `<p>` — Opening tag
- `Hello World` — Text content
- `</p>` — Closing tag

हे तिन्ही एकत्र मिळून paragraph element तयार करतात.

लक्षात ठेवण्याची पद्धत

Tag = Markup boundary

Element = Complete HTML construct

उदाहरण:

```
<h1>Introduction to HTML</h1>
```

इथे `<h1>` आणि `</h1>` हे tags आहेत. संपूर्ण heading म्हणजे element.

5.3 Void Elements

तुझ्या वहीत `<br>` हा उदाहरण म्हणून लिहिलेला दिसतो.

`<br>` हे HTML मधील void element आहे.

Void element मध्ये content ठेवण्यासाठी किंवा element बंद करण्यासाठी स्वतंत्र closing tag नसतो.

उदाहरण:

```
<p>Hello<br>World</p>
```

Browser मध्ये text दोन ओळींमध्ये दिसेल.

HTML मधील इतर काही void elements:

```
<img src="photo.jpg" alt="A photograph">

<hr>

<input type="text">

<meta charset="UTF-8">
```

Technical correction:

HTML मध्ये `<br/>` असे लिहिलेले दिसू शकते, परंतु HTML syntax मध्ये void elements साठी trailing slash आवश्यक नाही.

```
<br>
```

हे पुरेसे आहे.

Void elements मध्ये `img`, `br`, `hr`, `meta`, `link`, `input`, `area`, `base`, `col`, `embed`, `source`, `track` आणि `wbr` यांचा समावेश होतो.

---

6. HTML Document Structure

तुझ्या वहीत खालील tags दिसतात:

- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<title>`
- `<body>`
- `<h1>`
- `<p>`

यांचा वापर करून आपण HTML document ची मूलभूत रचना समजून घेऊ.

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Introduction to HTML</title>
</head>

<body>

    <h1>Welcome to HTML</h1>

    <p>This is my first web page.</p>

</body>

</html>
```

आता प्रत्येक भाग समजून घेऊ.

6.1 `<!DOCTYPE html>`

ही HTML document ची document type declaration आहे.

आधुनिक HTML मध्ये ती browser ला document HTML standard नुसार render करण्यास मदत करते.

मराठीत:

Browser ला document standards mode मध्ये render करायचा आहे हे सांगण्यासाठी ही declaration वापरली जाते.

```
<!DOCTYPE html>
```

महत्त्वाचे: ही HTML element नाही. ही document type declaration आहे.

6.2 `<html>`

हा HTML document चा root element आहे.

```
<html lang="en">
```

संपूर्ण HTML document ची मुख्य structure या element च्या आत असते.

`lang="en"` हे document ची मुख्य भाषा इंग्रजी आहे असे दर्शवते.

मराठी content साठी:

```
<html lang="mr">
```

योग्य language declaration accessibility आणि काही language-processing features साठी उपयुक्त असते.

6.3 `<head>`

`<head>` मध्ये document बद्दलची metadata आणि संबंधित resources ची माहिती असते.

उदाहरण:

```
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
</head>
```

`<head>` मधील content सामान्यतः Web page च्या मुख्य visible content मध्ये दाखवले जात नाही.

त्यात खालील गोष्टी असू शकतात:

- Character encoding
- Page title
- Viewport configuration
- SEO साठी उपयुक्त metadata
- CSS stylesheets चे links
- काही scripts आणि इतर document metadata

6.4 `<title>`

हा document चा title define करतो.

```
<title>My Website</title>
```

हा title browser tab मध्ये दिसतो आणि bookmarks मध्येही वापरला जाऊ शकतो.

Advanced insight:

Search engines अनेकदा page चा title search results मध्ये वापरतात. मात्र, ते नेहमी हाच title जसाच्या तसा दाखवतील याची हमी नसते.

6.5 `<body>`

`<body>` मध्ये document चा मुख्य content असतो.

उदाहरण:

```
<body>
    <h1>Welcome</h1>
    <p>This is my first paragraph.</p>
</body>
```

यात headings, paragraphs, images, links, forms आणि इतर visible page content असू शकतो.

6.6 `<h1>` आणि `<p>`

`<h1>`

मुख्य heading किंवा सर्वाधिक महत्त्वाची heading दर्शवण्यासाठी वापरला जातो.

```
<h1>Introduction to HTML</h1>
```

`<p>`

Paragraph दर्शवण्यासाठी वापरला जातो.

```
<p>HTML structures web content.</p>
```

Advanced insight:

HTML headings चा वापर फक्त text मोठा दाखवण्यासाठी करू नये. त्या document ची semantic hierarchy दर्शवतात. Visual appearance साठी CSS वापरणे योग्य आहे.

---

7. Metadata

तुझ्या वहीत `Metadata`, `<head>`, `SEO`, `UTF-8`, `3 byte` आणि `viewport → device / responsive` असे मुद्दे दिसतात.

हे महत्त्वाचे concepts आहेत.

7.1 Metadata म्हणजे काय?

Definition: Metadata is data that describes other data.

मराठीत:

Metadata म्हणजे document बद्दलची माहिती.

उदाहरणार्थ, एखाद्या Web page मध्ये मुख्य content म्हणून एखादा article आहे.

त्या article शी संबंधित metadata मध्ये खालील गोष्टी असू शकतात:

- Document चे title
- Character encoding
- Language
- Description
- Viewport configuration

HTML मध्ये metadata प्रामुख्याने `<head>` element मध्ये ठेवले जाते.

7.2 Character Encoding — UTF-8

तुझ्या वहीत `UTF-8` आणि `3 byte` अशी नोंद दिसते.

UTF-8 हे Unicode characters encode करण्यासाठी वापरले जाणारे variable-length character encoding आहे.

```
<meta charset="UTF-8">
```

या declaration मुळे browser ला document साठी UTF-8 encoding वापरायचे आहे हे समजते.

UTF-8 मध्ये एका Unicode code point साठी साधारणपणे 1 ते 4 bytes लागतात.

Character चा प्रकार	UTF-8 मधील bytes
अनेक सामान्य English characters	1 byte
काही Latin आणि इतर scripts मधील characters	2 bytes
अनेक भारतीय भाषांतील characters	3 bytes
काही इतर Unicode characters	4 bytes

महत्त्वाचे correction:

UTF-8 म्हणजे प्रत्येक character 3 bytes मध्ये store होतो असे नाही.

UTF-8 हे variable-length encoding आहे. Character नुसार 1, 2, 3 किंवा 4 bytes वापरले जातात.

7.3 Viewport

तुझ्या वहीत:

Viewport → Device / Responsive

अशी नोंद आहे.

Viewport म्हणजे browser मध्ये Web page चा content पाहण्यासाठी उपलब्ध असलेला visible area.

मोबाइल आणि desktop वर उपलब्ध viewport dimensions वेगवेगळे असू शकतात.

यासाठी HTML मध्ये खालील metadata वापरली जाते:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

यातील:

- `name="viewport"` — Viewport metadata दर्शवते.
- `width=device-width` — Layout viewport ची width device च्या width शी जुळवते.
- `initial-scale=1.0` — सुरुवातीचा zoom scale 1 ठेवते.

Real-life example:

समजा, तू एखादी website laptop वर उघडली आणि त्याच website ला mobile वर उघडले.

Responsive design योग्य प्रकारे काम करण्यासाठी viewport configuration आणि CSS media queries सारख्या तंत्रांचा वापर केला जातो.

महत्त्वाचे:

फक्त viewport meta tag लिहिल्याने website पूर्णपणे responsive होत नाही. त्यासाठी CSS layout, flexible dimensions, images आणि इतर responsive design techniques देखील आवश्यक असतात.

7.4 SEO — Search Engine Optimization

तुझ्या वहीत `SEO` हा शब्द दिसतो.

SEO म्हणजे Search Engine Optimization.

Search engines ला Web pages समजण्यास आणि योग्य संदर्भात दाखवण्यास मदत होईल अशा पद्धतीने website तयार करण्याच्या प्रक्रियेला SEO म्हणतात.

HTML च्या संदर्भात खालील गोष्टी महत्त्वाच्या आहेत:

- अर्थपूर्ण `<title>`
- योग्य heading hierarchy
- स्पष्ट आणि semantic HTML
- Links साठी अर्थपूर्ण text
- Images साठी योग्य `alt` text
- योग्य language declaration
- आवश्यकतेनुसार description metadata

उदाहरण:

```
<head>
    <title>Java Full Stack Development Course</title>

    <meta name="description"
          content="Learn Java, Spring Boot, HTML, CSS and JavaScript.">
</head>
```

Advanced insight:

SEO केवळ meta tags वर अवलंबून नसते. Content quality, accessibility, page performance, crawlability, links आणि इतर अनेक गोष्टी त्यावर प्रभाव टाकतात.

---

8. DOM — Document Object Model

तुझ्या वहीत `DOM Reference → Document` अशी नोंद दिसते.

DOM समजणे हे HTML आणि browser rendering समजण्यासाठी अत्यंत महत्त्वाचे आहे.

8.1 DOM म्हणजे काय?

Full form: Document Object Model.

Definition: The DOM is a programming interface that represents an HTML document as a tree of objects.

मराठीत:

Browser HTML document parse करून त्याचे object-based representation तयार करतो. या representation ला DOM म्हणतात.

JavaScript च्या मदतीने DOM मधील elements शोधता येतात आणि त्यांचा content, attributes किंवा काही properties बदलता येतात.

उदाहरण:

```
<html>
    <body>
        <h1>Hello</h1>
        <p>Welcome to HTML.</p>
    </body>
</html>
```

याची simplified DOM tree अशी दिसते:

```
Document
   |
  html
   |
  body
  /  \
 h1   p
 |    |
Hello Welcome to HTML.
```

या tree मध्ये:

- `Document` हा document चा entry point आहे.
- `html` हा root element आहे.
- `body` हा `html` चा child आहे.
- `h1` आणि `p` हे `body` चे children आहेत.

8.2 DOM चा उपयोग

उदाहरण:

```
<h1 id="heading">Hello</h1>
```

JavaScript:

```
const heading = document.getElementById("heading");

heading.textContent = "Welcome";
```

आता page वरील heading चा text बदलतो.

Advanced insight:

DOM आणि HTML source code एकच गोष्ट नाहीत.

HTML source ही document ची markup representation आहे. DOM हे browser ने तयार केलेले object model आहे.

Browser HTML parse करताना काही corrections किंवा implied elements तयार करू शकतो. त्यामुळे DOM आणि source code मध्ये काही फरक असू शकतात.

---

9. Browser HTML कसे Process करतो?

तुझ्या वहीतील Browser, HTML text, DOM, layout, paint, rasterization आणि CPU/GPU संबंधित diagram चा अर्थ समजून घेऊ.

हा browser rendering pipeline चा महत्त्वाचा भाग आहे.

9.1 Basic Flow

```
HTML Response
      |
      v
HTML Parsing
      |
      v
DOM Tree
      |
      v
CSS Processing
      |
      v
Style Calculation
      |
      v
Layout
      |
      v
Paint
      |
      v
Rasterization
      |
      v
Compositing
      |
      v
Rendered Page
```

ही एक simplified representation आहे. वास्तविक browser मध्ये काही stages overlap होऊ शकतात आणि rendering process अधिक गुंतागुंतीची असते.

Step 1 — HTML Parsing

Browser ला HTML bytes मिळतात.

Browser त्या bytes चे decoding करून characters तयार करतो आणि HTML parser markup process करतो.

Step 2 — DOM Construction

HTML parser elements आणि nodes यांची DOM tree तयार करतो.

Step 3 — CSS Processing

Browser CSS rules process करतो.

DOM मधील elements वर कोणते CSS rules लागू होतात हे ठरवण्यासाठी selector matching आणि style calculation केले जाते.

Step 4 — Layout

Elements चा आकार आणि page वर त्यांची position ठरवली जाते.

उदाहरण:

- Heading कुठे दिसेल?
- Paragraph किती जागा घेईल?
- Image ची dimensions किती असतील?

Step 5 — Paint

Browser elements चे visual representation तयार करण्यासाठी painting operations तयार करतो.

उदाहरण:

- Text render करणे
- Background रंग दाखवणे
- Borders काढणे

Step 6 — Rasterization

Painting instructions मधून pixels तयार केले जातात.

Browser architecture नुसार ही प्रक्रिया वेगवेगळ्या rendering components किंवा threads वर होऊ शकते.

Step 7 — Compositing

वेगवेगळ्या rendered layers एकत्र करून अंतिम दृश्य तयार केले जाते.

यामध्ये GPU चा उपयोग होऊ शकतो.

CPU आणि GPU

तुझ्या diagram मध्ये CPU आणि GPU हे शब्द दिसतात.

- **CPU:** HTML parsing, JavaScript execution, style calculation आणि layout यांसारख्या अनेक कामांमध्ये भूमिका बजावतो.
- **GPU:** काही graphics आणि compositing operations वेगाने करण्यासाठी वापरला जाऊ शकतो.

महत्त्वाचे: प्रत्येक rendering operation GPU वरच होते असे नाही. Browser, operating system आणि rendering workload नुसार कामाचे विभाजन बदलते.

---

10. Browser, Rendering Engine आणि JavaScript Engine

तुझ्या वहीत Chrome आणि CSS संबंधित नोंदी दिसतात.

हे समजण्यासाठी खालील distinction लक्षात ठेव.

Browser

Browser हे user-facing application आहे.

उदाहरण:

- Google Chrome
- Mozilla Firefox
- Safari

Browser मध्ये अनेक components असतात, जसे:

- Rendering engine
- JavaScript engine
- Networking components
- Storage components
- Browser UI

Rendering Engine

Rendering engine HTML आणि CSS process करून Web page चे visual representation तयार करण्यात महत्त्वाची भूमिका बजावतो.

उदाहरणार्थ, Chrome चा rendering engine Blink आहे.

JavaScript Engine

JavaScript code execute करण्यासाठी JavaScript engine वापरला जातो.

उदाहरणार्थ, Chrome मध्ये V8 JavaScript engine वापरला जातो.

Interview Question

Q. Is Chrome the same as Blink?

Answer:

No. Chrome is a Web browser, whereas Blink is its rendering engine. Chrome also includes other components, including the V8 JavaScript engine and networking infrastructure.

---

11. Document Structure आणि Tree Representation

तुझ्या शेवटच्या diagram मध्ये खालील concepts दिसतात:

- Document
- Doctype
- Head
- Title
- Body
- Heading
- Paragraph
- Text

यातून HTML ची hierarchical structure समजते.

उदाहरण:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <title>My Page</title>
</head>

<body>
    <h1>Introduction</h1>

    <p>HTML is a markup language.</p>
</body>

</html>
```

Simplified tree:

```
Document
 |
 html
 /  \
head body
 |    / \
title h1 p
       | |
     Text Text
```

हे diagram समजून घे.

`html` element चे दोन महत्त्वाचे children म्हणजे `head` आणि `body`.

`head` मध्ये document metadata असते.

`body` मध्ये page चा मुख्य content असतो.

`body` मधील `h1` आणि `p` यांना text content असू शकतो.

DOM tree समजल्यामुळे JavaScript, CSS selectors, accessibility आणि browser rendering समजणे सोपे होते.

---

12. आजच्या Lecture मधील महत्त्वाचे Interview Questions

Q1. What is HTML?

HTML is the standard markup language used to structure content on the Web.

Q2. Why is HTML not a programming language?

HTML describes the structure and semantics of content. It does not provide general-purpose programming constructs such as loops and functions in the way Java or Python does.

Q3. What is the difference between HTML and the DOM?

HTML is a markup language used to describe document structure. The DOM is an object-based representation of the document that scripts and browser APIs can interact with.

Q4. What is the purpose of `<!DOCTYPE html>`?

It declares the document type and helps browsers render the page in standards mode.

Q5. What is metadata?

Metadata is information about a document, such as its title, character encoding and viewport configuration.

Q6. What is UTF-8?

UTF-8 is a variable-length Unicode character encoding that uses one to four bytes per Unicode code point.

Q7. What is the purpose of the viewport meta tag?

It configures the layout viewport and initial scale, helping pages display appropriately on different devices.

Q8. What is the difference between a tag and an element?

A tag is a markup construct such as `<p>` or `</p>`. An element is the complete HTML construct, including its tags and content where applicable.

Q9. What is a void element?

A void element is an HTML element that cannot contain child content and does not have a closing tag, such as `<br>` or `<img>`.

Q10. What happens when a browser receives HTML?

The browser decodes and parses the HTML, constructs a DOM, processes CSS, calculates layout, paints content and composites the rendered output. The exact internal sequence can vary by browser and workload.

---

13. आजच्या Lecture ची Quick Revision

Concept	लक्षात ठेवण्याची गोष्ट
UNIX	Operating System
ARPANET	Internet च्या इतिहासातील महत्त्वाचे precursor
WWW	Interlinked Web resources ची व्यवस्था
CERN	Web च्या सुरुवातीच्या विकासाशी संबंधित संशोधन संस्था
Tim Berners-Lee	World Wide Web चे प्रस्तावक आणि प्रमुख निर्माते
HTML	Web content ची structure define करते
CSS	Presentation आणि styling नियंत्रित करते
JavaScript	Dynamic behavior आणि interactions तयार करते
Tag	HTML markup चा भाग
Element	HTML document मधील संपूर्ण construct
Metadata	Document बद्दलची माहिती
UTF-8	Unicode साठी variable-length encoding
Viewport	Web page पाहण्यासाठी उपलब्ध visible area
DOM	HTML document चे object-based representation
Layout	Elements चा आकार आणि position ठरवणे
Paint	Visual representation तयार करण्यासाठी painting operations
Rasterization	Graphics चे pixels मध्ये रूपांतर
Compositing	Rendered layers एकत्र करणे

महत्त्वाची नोंद — तुझ्या वहीतील अस्पष्ट भाग

मी वाचता आलेल्या technical terms आणि diagrams समाविष्ट केले आहेत. मात्र, पहिल्या फोटोतील मराठी वाक्य, दुसऱ्या फोटोतील काही जुनी historical notes आणि काही लहान handwritten labels पूर्णपणे स्पष्ट नाहीत. त्यांचा शब्दशः मजकूर निश्चितपणे वाचता येत नसल्यामुळे मी त्यांना अचूक transcription म्हणून मांडलेले नाही.

तसेच, या उत्तरात काही technical explanations संदर्भ समजण्यासाठी विस्ताराने दिली आहेत; त्या प्रत्येक गोष्टी शिक्षकांनी आजच्या lecture मध्ये शिकवल्या होत्या, असा दावा नाही.

उद्याच्या lecture मध्ये HTML tags शिकताना आजची document structure, metadata, tag vs element आणि DOM यांची समज तुला उपयोगी पडेल.....Santosh, आता मला तुझी अपेक्षा स्पष्ट समजली. तुला फक्त HTML च्या definitions किंवा tags ची यादी नको आहे. तुला **HTML, Browser, DOM, C++ मधील Classes आणि Objects, आणि browser प्रत्यक्षात काय करतो हे internals पासून समजून घ्यायचे आहे.** त्याचबरोबर, एखाद्या experienced developer प्रमाणे वेगवेगळ्या परिस्थितींमध्ये काय होईल, हे स्वतःच्या reasoning ने सांगता आले पाहिजे.

तुझ्या आधीच्या notes मध्ये काही महत्त्वाचे edge cases राहिले होते. ते आता समजून घेऊ.

HTML Introduction — Advanced Engineering Notes

Part 1: HTML, C++ Classes, DOM Objects, Browser Internals and Edge Cases

या notes मध्ये:

- **Core concept:** नेमके काय आहे?
- **मराठीत explanation:** ते कसे काम करते?
- **Edge cases:** वेगवेगळ्या परिस्थितींमध्ये काय होते?
- **Engineering insight:** अनुभवी developer ने काय समजून घेतले पाहिजे?

---

1. सर्व HTML Tags हे C++ Classes आहेत का?

तू म्हणालास की सर्व HTML tags या C++ classes आहेत आणि DOM tree मध्ये classes व objects असतात.

या विधानामध्ये एक महत्त्वाचा भाग आहे, पण एक correction आवश्यक आहे.

HTML tags स्वतः C++ classes नाहीत. मात्र, browser च्या implementation मध्ये HTML elements आणि DOM objects यांच्यासाठी C++ classes वापरल्या जाऊ शकतात.

हे समजून घेण्यासाठी C++ मधील OOP चे basics पाहू.

1.1 C++ मधील Class आणि Object

समजा, आपण C++ मध्ये एक class तयार केली.

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;

    void display() {
        cout << name;
    }
};

int main() {
    Student s1;

    s1.name = "Santosh";
    s1.display();

    return 0;
}
```

इथे:

- `Student` ही class आहे.
- `s1` हा object आहे.
- `name` हे data member आहे.
- `display()` हे member function आहे.

Class हा object तयार करण्यासाठीचा आराखडा असतो.

Object हा त्या class चा instance असतो.

1.2 आता HTML शी संबंध जोडू

खालील HTML पाहा.

```
<h1>Hello World</h1>
<p>Welcome to HTML</p>
```

Browser हा HTML text parse करतो आणि त्याचे DOM representation तयार करतो.

Conceptually, JavaScript मधून आपण असे objects पाहू शकतो:

```
const heading = document.querySelector("h1");

console.log(heading);
console.log(heading.constructor.name);
```

Browser मध्ये `heading` हा DOM element object आहे.

सामान्य browsers मध्ये `<h1>` element चा DOM interface `HTMLHeadingElement` असतो, जो `HTMLElement` आणि पुढे `Element` व `Node` यांसारख्या interfaces शी संबंधित असतो.

तसेच `<p>` साठी `HTMLParagraphElement` हा interface वापरला जातो.

महत्त्वाचे: हे JavaScript मधून दिसणारे Web API interfaces आहेत. त्यांचा संबंध browser च्या internal C++ implementation शी असू शकतो, परंतु DOM specification प्रत्येक browser ने C++ classes चाच वापर करावा अशी अट घालत नाही.

Browser ची implementation language वेगळी असू शकते.

उदाहरणार्थ, Chromium मध्ये Blink rendering engine ची मोठी implementation C++ मध्ये आहे.

Conceptual relationship

```
HTML Source
     |
     v
HTML Parser
     |
     v
DOM Element Object
     |
     v
Browser Rendering System
```

HTML source मधील `<h1>` हे markup आहे.

त्यातून तयार होणारा `HTMLHeadingElement` हा DOM object आहे.

या object वर properties आणि methods वापरून JavaScript मधून काम करता येते.

---

2. DOM Tree मध्ये Classes आणि Objects कसे असतात?

समजा, तुझ्याकडे हा HTML document आहे.

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>

<body>
    <h1>Hello</h1>
    <p>Welcome</p>
</body>
</html>
```

Browser याचे DOM tree तयार करतो.

```
Document
   |
   html
   |
   +---- head
   |       |
   |      title
   |       |
   |    "My Website"
   |
   +---- body
           |
           +---- h1
           |      |
           |    "Hello"
           |
           +---- p
                  |
               "Welcome"
```

प्रत्येक element हा DOM मधील object म्हणून represent होतो.

JavaScript मधून:

```
const heading = document.querySelector("h1");
const paragraph = document.querySelector("p");

console.log(heading);
console.log(paragraph);
```

`heading` आणि `paragraph` हे वेगवेगळे DOM objects आहेत.

त्यांचे interfaces वेगवेगळे असू शकतात, कारण त्यांची semantic roles वेगवेगळी आहेत.

उदाहरण

```
console.log(heading instanceof Element);
console.log(heading instanceof HTMLElement);
console.log(heading instanceof HTMLHeadingElement);
```

सामान्य browser environment मध्ये या तिन्ही expressions चे उत्तर `true` असेल.

याचे कारण DOM interfaces मधील inheritance relationship आहे.

Conceptually:

```
EventTarget
    |
   Node
    |
  Element
    |
 HTMLElement
    |
HTMLHeadingElement
```

हा simplified interface hierarchy चा भाग आहे.

Experienced developer ने काय लक्षात ठेवावे?

HTML element चे DOM object हे फक्त text representation नसते.

त्यावर:

- Properties असतात.
- Methods असतात.
- Events attach करता येतात.
- Attributes वाचता आणि बदलता येतात.
- DOM tree मधील parent-child relationships वापरता येतात.

म्हणून DOM समजण्यासाठी OOP ची basic समज उपयोगी ठरते.

---

3. आपण फक्त `<body>` Tag लिहिला तर काय होते?

हा खूप चांगला edge case आहे.

समजा, तुझ्या HTML file मध्ये फक्त हे आहे:

```
<body>
    <h1>Hello World</h1>
    <p>This is HTML.</p>
</body>
```

तू ही file browser मध्ये उघडली.

तर काय होईल?

सामान्य browser HTML document म्हणून हा मजकूर parse करेल. त्यात `<html>`, `<head>` आणि इतर काही structural elements स्पष्टपणे नसले तरी HTML parser आवश्यकतेनुसार document ची structure तयार करू शकतो.

Conceptually तयार होणारा document:

```
<html>
<head></head>

<body>
    <h1>Hello World</h1>
    <p>This is HTML.</p>
</body>

</html>
```

हे समजून घेणे महत्त्वाचे आहे.

Browser हे कसे करतो?

HTML parsing साठी browser मध्ये defined parsing rules असतात.

जेव्हा आवश्यक structural elements source मध्ये नसतात, तेव्हा HTML parser काही elements implicit स्वरूपात तयार करू शकतो.

याला parser द्वारा केलेली error recovery आणि implied element insertion यांसारख्या प्रक्रियांशी संबंधित समजता येते.

म्हणजे HTML file मध्ये `<html>` आणि `<head>` लिहिले नाहीत म्हणून browser लगेच fail होत नाही.

परंतु याचा अर्थ असा नाही की production application मध्ये पूर्ण document structure टाळावी.

Experienced developer काय करेल?

Production मध्ये शक्यतो असा स्पष्ट document लिहील:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>My Website</title>
</head>

<body>

    <h1>Hello World</h1>

</body>

</html>
```

यामुळे document maintainable राहतो आणि metadata, accessibility, responsive behavior यांसाठी आवश्यक configuration स्पष्ट होते.

---

4. आपण `<!DOCTYPE html>` लिहिले नाही तर काय होते?

आता दुसरी परिस्थिती पाहू.

Case A — DOCTYPE आहे

```
<!DOCTYPE html>

<html>
<body>
    <h1>Hello</h1>
</body>
</html>
```

आधुनिक browsers सामान्यतः standards mode मध्ये page render करतात.

Case B — DOCTYPE नाही

```
<html>
<body>
    <h1>Hello</h1>
</body>
</html>
```

अशा document मध्ये अनेक browsers quirks mode मध्ये जाऊ शकतात.

Quirks Mode म्हणजे काय?

Quirks mode हा जुन्या Web pages शी compatibility राखण्यासाठी browsers मध्ये असलेला rendering mode आहे.

पूर्वीच्या browsers मध्ये काही layout आणि CSS behaviors आधुनिक standards पेक्षा वेगळे होते.

जुन्या websites तुटू नयेत म्हणून browsers काही legacy behaviors support करतात.

Standards Mode vs Quirks Mode

मुद्दा	Standards Mode	Quirks Mode
उद्देश	आधुनिक Web standards नुसार rendering	जुन्या pages सोबत compatibility
Layout behavior	आधुनिक standards च्या अधिक जवळ	काही legacy CSS behaviors
नवीन projects	योग्य पर्याय	शक्यतो टाळावा
DOCTYPE	योग्य declaration असल्यास सामान्यतः	काही declarations नसल्यास किंवा विशिष्ट परिस्थितीत

महत्त्वाची गोष्ट

फक्त DOCTYPE नसल्यामुळे HTML document चा DOM तयार होणे बंद होत नाही.

DOM तयार होऊ शकतो; मुख्य फरक rendering mode आणि काही layout behaviors मध्ये असतो.

Interview answer:

`<!DOCTYPE html>` tells the browser to use standards mode for an HTML document. Omitting it can trigger quirks mode, which exists mainly for compatibility with legacy Web content.

---

5. जर आपण `<title>` लिहिलेच नाही तर Browser Tab वर काय दिसेल?

हीदेखील महत्त्वाची edge case आहे.

Case A — Title दिला आहे

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <h1>Hello</h1>
</body>
</html>
```

Browser tab मध्ये सामान्यत::

`My Website`

दिसेल.

Case B — Title नाही

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
    <h1>Hello</h1>
</body>
</html>
```

आता काय होईल?

HTML document मध्ये `<title>` element नसला तरी page render होऊ शकतो.

परंतु document ला लेखकाने स्पष्टपणे दिलेला title उपलब्ध नसतो.

Browser tab, window title किंवा browser UI मध्ये काय दाखवले जाईल हे browser आणि environment वर अवलंबून असते. काही browsers URL, filename किंवा इतर fallback information वापरू शकतात.

म्हणून प्रत्येक browser मध्ये tab रिकामा असेल किंवा नेमकाच filename दिसेल, असे निश्चितपणे म्हणता येत नाही.

Experienced developer काय करेल?

नेहमी meaningful title देईल.

```
<title>Java Full Stack Learning Portal</title>
```

कारणे:

1. Browser tab मध्ये page ओळखता येतो.
2. अनेक tabs उघडे असताना navigation सोपे होते.
3. Search engines title वापरू शकतात.
4. Screen reader users ना page ओळखण्यासाठी मदत होते.

---

6. `<head>` नसल्यास काय होईल?

उदाहरण:

```
 <!DOCTYPE html>
<html>
<title>My Website</title>

<body>
    <h1>Hello World</h1>
</body>
</html>
```

इथे `<head>` स्पष्टपणे लिहिलेला नाही.

तरीही browser document parse करू शकतो.

HTML parsing rules नुसार काही missing structural elements तयार केले जाऊ शकतात आणि `<title>` सारखा metadata element योग्य document location मध्ये place केला जाऊ शकतो.

म्हणून source code आणि final DOM मध्ये फरक असू शकतो.

एक महत्त्वाची गोष्ट

HTML parser हा फक्त प्रत्येक opening tag च्या बदल्यात एक object तयार करणारा साधा text processor नाही.

तो HTML parsing algorithm नुसार काम करतो.

त्यात:

- Tokenization
- Tree construction
- Insertion modes
- Error recovery
- Implied elements
- Misnested tags चे handling

यांसारख्या गोष्टींचा समावेश असतो.

या कारणामुळे invalid किंवा incomplete HTML दिले तरी browser बऱ्याचदा usable DOM तयार करू शकतो.

---

7. HTML Source आणि DOM यांमध्ये काय फरक आहे?

हा experienced frontend आणि browser-engineering interviews साठी महत्त्वाचा विषय आहे.

समजा, तुझा source code असा आहे:

```
<p>Hello
<p>World
```

इथे closing `</p>` tags दिलेले नाहीत.

तरीही browser दोन्ही paragraphs स्वतंत्रपणे दाखवू शकतो.

HTML parsing rules नुसार नवीन `<p>` element सुरू झाल्यावर आधीचा paragraph बंद केला जाऊ शकतो.

Conceptually DOM असा असू शकतो:

```
body
 |
 +---- p
 |      |
 |    "Hello"
 |
 +---- p
        |
      "World"
```

निष्कर्ष

Browser चा DOM हा source code ची अक्षरशः copy नसतो.

तो HTML parsing rules नुसार तयार झालेला document model असतो.

दुसरे उदाहरण

Source:

```
<table>
    <tr>
        <td>Java</td>
    </tr>
</table>
```

Browser DOM मध्ये table structure साठी implicit `<tbody>` element तयार करू शकतो.

Conceptually:

```
table
  |
 tbody
  |
  tr
  |
  td
  |
 "Java"
```

त्यामुळे browser DevTools मधील Elements panel मध्ये असा `<tbody>` दिसू शकतो, जो तू source मध्ये स्पष्टपणे लिहिलेला नसतो.

---

8. जर आपण फक्त `<html>` लिहिले तर?

उदाहरण:

```
<html>
</html>
```

यामध्ये `head` किंवा `body` स्पष्टपणे दिलेले नाहीत.

तरीही browser HTML parsing rules नुसार document structure तयार करू शकतो.

परंतु visible content नसल्यामुळे page वर काहीही विशेष दिसणार नाही.

त्याचप्रमाणे:

```
<h1>Hello</h1>
```

असे लिहिले तरी browser सामान्य HTML parsing context मध्ये आवश्यक structure तयार करून heading दाखवू शकतो.

महत्त्वाचे: हे HTML documents browser ने कसे recover करावेत याचे वर्णन आहे; पूर्ण document structure टाळण्याची शिफारस नाही.

---

9. Browser ला HTML File कशी मिळते?

आता वास्तविक Web architecture समजून घेऊ.

जेव्हा तू एखादी website उघडतोस, तेव्हा साधारणपणे खालील प्रक्रिया होते.

```
User enters URL
       |
       v
Browser processes URL
       |
       v
DNS resolution, if needed
       |
       v
Connection establishment
       |
       v
HTTP request
       |
       v
Server sends HTTP response
       |
       v
HTML bytes received
       |
       v
HTML parsing
       |
       v
DOM construction
       |
       v
CSS, JavaScript and resources
       |
       v
Rendering
       |
       v
Web page displayed
```

हा एक simplified flow आहे.

वास्तविक परिस्थितीत browser cache वापरू शकतो, connection आधीपासून उपलब्ध असू शकते, JavaScript इतर resources dynamically load करू शकते आणि parsing व rendering मध्ये अनेक गोष्टी एकमेकांसोबत होऊ शकतात.

Real-life example

समजा, तू Spring Boot application deploy केले आहेस.

Browser ला request पाठवली:

```
GET /products HTTP/1.1
Host: example.com
```

Server HTTP response मध्ये HTML पाठवू शकतो.

किंवा Spring Boot backend JSON response देऊ शकतो आणि frontend JavaScript त्या JSON वरून DOM update करू शकते.

HTML आणि backend एकच गोष्ट नाहीत.

HTML हा frontend document structure चा भाग आहे. Backend data, authentication, business logic आणि इतर server-side operations हाताळू शकतो.

---

10. आता तुला C++ शिकण्याची किती गरज आहे?

तू सांगितलेस की तुला C++ बद्दल फारशी समज नाही.

काळजी करू नकोस. HTML शिकण्यासाठी C++ आवश्यक नाही.

तुला सध्या C++ चे खालील concepts समजून घेणे पुरेसे आहे:

1. Class म्हणजे काय?
2. Object म्हणजे काय?
3. Properties आणि methods म्हणजे काय?
4. Inheritance म्हणजे काय?
5. Interface किंवा abstraction म्हणजे काय?
6. Tree data structure म्हणजे काय?

हे concepts समजले की DOM ची रचना आणि browser internals समजणे सोपे जाईल.

पण लक्षात ठेव:

Browser internals समजण्यासाठी C++ च्या implementation details आणि HTML वापरण्यासाठी आवश्यक असलेले knowledge हे दोन वेगवेगळे स्तर आहेत.

HTML developer म्हणून काम करण्यासाठी तुला browser चे सर्व C++ source code समजणे आवश्यक नाही.

मात्र, rendering, DOM, event handling आणि performance समजल्यामुळे तू अधिक चांगला engineer होऊ शकतोस.

---

11. MDN Documentation कसे वाचायचे?

तू MDN documentation कशी वाचायची हे देखील विचारले आहेस.

MDN म्हणजे Mozilla Developer Network. Web development मधील HTML, CSS, JavaScript आणि Web APIs समजण्यासाठी हे एक महत्त्वाचे documentation resource आहे.

Official resource: MDN Web Docs

HTML शिकताना खालील क्रम वापर.

Step 1 — HTML Introduction

Introduction to HTML

इथे HTML काय आहे आणि Web content कसे structure करतात हे समजून घे.

Step 2 — HTML Elements

HTML Elements Reference

प्रत्येक element साठी खालील गोष्टी पाहा:

- Element चा उद्देश

- Syntax
- Attributes
- Content model
- Browser behavior
- संबंधित उदाहरणे

Step 3 — DOM

Document Object Model (DOM)

इथे HTML source आणि DOM representation यांतील फरक समजून घे.

Step 4 — HTML Parsing

Parsing HTML documents

हे अधिक advanced reference आहे.

यात HTML parser कसा काम करतो, incomplete किंवा malformed HTML कसे handle केले जाते आणि DOM कसा तयार होतो याची सविस्तर माहिती मिळते.

Step 5 — HTML Standard

WHATWG HTML Living Standard

हे HTML चे सविस्तर technical standard आहे.

सुरुवातीला संपूर्ण specification वाचण्याची गरज नाही. एखादा specific behavior समजून घ्यायचा असेल तेव्हा संबंधित section वाच.

---

12. MDN वाचताना कोणत्या गोष्टी लक्षात ठेवायच्या?

समजा, तू `<title>` ची documentation उघडली.

फक्त पहिली definition वाचून थांबू नकोस.

खालील प्रश्न स्वतःला विचार:

1. या element चा purpose काय आहे?
2. तो document मध्ये कुठे वापरला जातो?
3. तो नसेल तर काय होते?
4. चुकीच्या ठिकाणी वापरला तर काय होते?
5. Browser त्याला कसे interpret करतो?
6. त्याचा DOM शी काय संबंध आहे?
7. Accessibility वर त्याचा काय परिणाम होतो?
8. SEO किंवा performance शी त्याचा काही संबंध आहे का?
9. वेगवेगळ्या browsers मध्ये कोणते behavior बदलू शकते?
10. Production application मध्ये कोणती काळजी घ्यावी?

ही पद्धत वापरल्यास तू documentation मधून फक्त syntax नाही, तर engineering reasoning देखील शिकशील.

---

13. Experienced Developer ची HTML Checklist

एखादा HTML document तपासताना खालील गोष्टी पाहता आल्या पाहिजेत.

विषय	तपासायची गोष्ट
Document declaration	`<!DOCTYPE html>` आहे का?
Language	`<html lang="...">` योग्य आहे का?
Character encoding	UTF-8 declaration योग्य ठिकाणी आहे का?
Page title	Meaningful `<title>` आहे का?
Viewport	Mobile viewport configuration आहे का?
Semantics	योग्य HTML elements वापरले आहेत का?
Accessibility	Images साठी योग्य `alt` text आहे का?
DOM	Source आणि parsed DOM यांतील फरक समजतो का?
CSS	Layout responsive आहे का?
Performance	अनावश्यक resources किंवा rendering delays आहेत का?
Browser behavior	Missing किंवा invalid markup चा परिणाम समजतो का?
Standards	HTML specification नुसार markup योग्य आहे का?

---

14. आजच्या अभ्यासासाठी Practical Experiment

खालील चार files तयार करून browser मध्ये उघड.

Experiment A — Complete HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Experiment A</title>
</head>
<body>
    <h1>Hello World</h1>
</body>
</html>
```

Experiment B — DOCTYPE नाही

```
<html>
<head>
    <title>Experiment B</title>
</head>
<body>
    <h1>Hello World</h1>
</body>
</html>
```

Experiment C — फक्त Body

```
<body>
    <h1>Hello World</h1>
</body>
```

Experiment D — Title नाही

```
<!DOCTYPE html>
<html>
<head></head>
<body>
    <h1>Hello World</h1>
</body>
</html>
```

प्रत्येक file साठी:

1. Browser मध्ये उघड.
2. Page वर काय दिसते ते पाहा.
3. Tab title पाहा.
4. Developer Tools उघड.
5. Elements panel मध्ये DOM पाहा.
6. `document.compatMode` तपास.
7. Source code आणि DOM मधील फरक शोध.

JavaScript Console मध्ये:

```
console.log(document.compatMode);
console.log(document.title);
console.log(document.documentElement);
console.log(document.body);
```

`document.compatMode` मध्ये सामान्यत::

- `"CSS1Compat"` म्हणजे standards mode.
- `"BackCompat"` म्हणजे quirks mode.

हा practical तुला browser चे actual behavior समजून घेण्यासाठी खूप उपयोगी ठरेल.

शेवटचा महत्त्वाचा मुद्दा

तुला प्रत्येक HTML tag पाठ करण्यापेक्षा खालील chain समजून घ्यायची आहे:

HTML Source → Parser → DOM Objects → CSS Processing → Layout → Paint → Compositing → Display

ही chain समजली की HTML, DOM, CSS आणि browser rendering यांचा परस्पर संबंध स्पष्ट होतो.

आणि प्रत्येक नवीन tag शिकताना त्याचे syntax पाठ करण्याऐवजी हे विचार:

"हा tag नसल्यास काय होईल? चुकीच्या ठिकाणी वापरल्यास काय होईल? Browser त्याला कसे parse करेल? DOM मध्ये काय तयार होईल? आणि production मध्ये हा element वापरताना कोणत्या गोष्टींची काळजी घ्यावी?"

हीच पद्धत तुला basic HTML शिकण्यापासून मजबूत engineering understanding पर्यंत घेऊन जाईल.....# HTML Industry-Level Tricky Scenarios — 3+ Years Engineer

आज आपण HTML ची फक्त syntax शिकणार नाही. आपण browser ला एखादा HTML document दिल्यावर प्रत्यक्षात काय होते, DOM कसा तयार होतो, missing tags कसे handle होतात, आणि production मध्ये कोणत्या चुका होऊ शकतात हे scenarios मधून समजून घेणार आहोत.

प्रत्येक scenario मध्ये चार गोष्टी असतील:

- Scenario: प्रत्यक्षात काय situation आहे?
- Predict: Browser काय करेल?
- Reasoning: असे का होते?
- Industry insight: Experienced developer ने काय लक्षात ठेवावे?

Level 1 — Tricky HTML Fundamentals

Scenario 1: फक्त `<body>` tag लिहिला तर?

```
<body>
    <h1>Hello</h1>
    <p>Welcome to HTML</p>
</body>
```

Predict the result

Browser page render करेल का?

A. Browser error देईल आणि page दिसणार नाही.

B. Browser HTML parsing rules वापरून page render करू शकतो.

C. फक्त body tag दिसेल.

Engineering insight: Browser parser error-recovery rules वापरतो. तरीही production मध्ये complete document structure लिहिणे योग्य आहे.

Scenario 2: `<!DOCTYPE html>` नसल्यास काय होते?

```
<html>
<head>
    <title>Test</title>
</head>
<body>
    <div class="box">Hello</div>
</body>
</html>
```

उत्तर: Browser document render करू शकतो, परंतु तो quirks mode मध्ये जाऊ शकतो.

JavaScript Console मध्ये तपास:

```
console.log(document.compatMode);
```

- `CSS1Compat` — standards mode
- `BackCompat` — quirks mode

Industry insight: जुन्या websites मध्ये quirks mode मुळे layout differences दिसू शकतात. Modern projects मध्ये `<!DOCTYPE html>` वापर.

Scenario 3: `<title>` नसल्यास काय होते?

```
<!DOCTYPE html>
<html>
<head></head>
<body>
    <h1>Dashboard</h1>
</body>
</html>
```

Predict: Page दिसेल का? Browser tab रिकामा असेल का?

उत्तर: Page दिसू शकतो. पण document ला स्पष्ट title दिलेला नाही. Browser tab मध्ये काय दाखवले जाईल हे browser आणि environment वर अवलंबून असते.

```
console.log(document.title);
```

Title नसल्यास `document.title` सामान्यतः रिकामी string देतो.

Industry insight: प्रत्येक page ला meaningful title दे. उदाहरणार्थ, `Student Dashboard | Atlas` हे `Home` पेक्षा अधिक descriptive असू शकते.

Scenario 4: HTML मध्ये closing tag नसल्यास?

```
<!DOCTYPE html>
<html>
<body>
    <p>First paragraph
    <p>Second paragraph
</body>
</html>
```

Predict: दोन्ही paragraphs एकाच line मध्ये येतील का?

उत्तर: Browser HTML parsing rules नुसार दुसरा `<p>` सुरू होताना पहिला paragraph बंद करू शकतो. त्यामुळे दोन स्वतंत्र paragraphs तयार होतात.

Conceptual DOM:

```
body
 ├── p
 │    └── "First paragraph"
 └── p
      └── "Second paragraph"
```

Industry insight: Browser च्या recovery वर अवलंबून राहू नकोस. Code readable आणि maintainable ठेवण्यासाठी आवश्यक closing tags स्पष्टपणे लिही.

Scenario 5: `<br>` आणि `<br/>` यांमध्ये फरक आहे का?

```
<p>Hello<br>World</p>
```

आणि

```
<p>Hello<br/>World</p>
```

उत्तर: HTML syntax मध्ये दोन्ही सामान्यतः line break देतात. HTML मध्ये `<br>` हा void element आहे. त्याला closing tag लागत नाही.

```
<br></br>
```

हे मात्र योग्य HTML pattern नाही. Parser अतिरिक्त `<br>` तयार करू शकतो; त्यावर अवलंबून राहू नये.

Industry insight: HTML आणि XHTML चे syntax rules एकसारखे नाहीत. JSX मध्ये `<br />` सामान्यतः लिहिले जाते, पण ते HTML source syntax शी तंतोतंत समान नियमांवर चालत नाही.

Level 2 — DOM आणि Browser Internals

Scenario 6: Source मध्ये `<tbody>` नाही, पण DevTools मध्ये दिसतो!

```
<table>
    <tr>
        <td>Java</td>
    </tr>
</table>
```

तू Chrome DevTools च्या Elements panel मध्ये पाहतोस:

```
<table>
    <tbody>
        <tr>
            <td>Java</td>
        </tr>
    </tbody>
</table>
```

असे का झाले?

HTML parser table parsing rules नुसार implicit `<tbody>` element तयार करू शकतो.

म्हणून HTML source आणि DOM representation नेहमी अक्षरशः सारखे नसतात.

Industry insight: Bug शोधताना फक्त View Page Source वर अवलंबून राहू नकोस. DevTools मधील Elements panel आणि actual DOM तपास.

Scenario 7: HTML tags म्हणजे C++ classes आहेत का?

समजा:

```
<h1>Hello</h1>
<p>World</p>
```

आणि JavaScript:

```
const heading = document.querySelector("h1");
const paragraph = document.querySelector("p");

console.log(heading instanceof HTMLElement);
console.log(paragraph instanceof HTMLElement);
```

दोन्ही results सामान्य browser मध्ये `true` असतील.

हे का?

दोन्ही DOM elements हे `HTMLElement` interface च्या inheritance hierarchy मध्ये येतात.

पण लक्षात ठेव:

- HTML tag म्हणजे C++ class नाही.
- DOM element हा browser मधील object आहे.

- Chromium सारखे browsers त्यांच्या internals मध्ये C++ वापरू शकतात.
- Web standards DOM behavior define करतात; ते C++ implementation अनिवार्य करत नाहीत.

Industry insight: Specification काय सांगते आणि एखादा browser ते internally कसे implement करतो, यांमध्ये फरक ओळखता आला पाहिजे.

Scenario 8: `innerHTML` आणि `textContent` वेगळे कसे वागतात?

```
<div id="demo"></div>
```

JavaScript:

```
const demo = document.getElementById("demo");

demo.innerHTML = "<strong>Hello</strong>";
```

Result: `Hello` bold दिसेल.

आता:

```
demo.textContent = "<strong>Hello</strong>";
```

Result: `<strong>Hello</strong>` हा text अक्षरशः दिसेल.

का?

- `innerHTML` HTML markup parse करून DOM मध्ये insert करते.
- `textContent` text content म्हणून value सेट करते.

Security scenario: User कडून आलेला untrusted input `innerHTML` मध्ये थेट घातल्यास Cross-Site Scripting (XSS) vulnerability निर्माण होऊ शकते.

सामान्य text दाखवण्यासाठी `textContent` हा अधिक सुरक्षित पर्याय आहे.

Scenario 9: JavaScript ने DOM बदलला तर HTML file बदलते का?

Original HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript:

```
document.getElementById("title").textContent = "Welcome";
```

आता browser मध्ये `Welcome` दिसेल.

पण मूळ `.html` file आपोआप बदलत नाही.

कारण: JavaScript ने in-memory DOM बदलला आहे; disk वरील source file नाही.

Industry insight: DevTools मधील DOM बदल पाहून source file बदलली असे समजू नये. Persist करण्यासाठी application logic किंवा file-writing process आवश्यक असतो.

Scenario 10: DOM element आहे, पण तो page वर दिसत नाही. का?

```
<p id="message" style="display: none;">
    Hello
</p>
```

```
console.log(document.getElementById("message"));
```

Element मिळतो, तरी page वर दिसत नाही.

कारण: DOM मध्ये असणे आणि visually rendered होणे या वेगळ्या गोष्टी आहेत.

Element hidden असू शकतो, CSS मुळे invisible असू शकतो, viewport बाहेर असू शकतो किंवा इतर element च्या मागे असू शकतो.

उदाहरणार्थ:

- `display: none` — सामान्यतः layout मधूनही element काढतो.
- `visibility: hidden` — जागा ठेवतो पण element दिसत नाही.
- `opacity: 0` — element transparent असतो; तरी तो काही interactions साठी उपलब्ध असू शकतो.

Industry insight: UI bug debug करताना DOM, computed CSS, layout आणि accessibility behavior वेगवेगळे तपास.

Level 3 — Production आणि Security Scenarios

Scenario 11: User ने HTML मध्ये malicious content दिले तर?

समजा, एखाद्या comment field मधून user ने हा input दिला:

```
<img src="invalid" onerror="alert('XSS')">
```

आणि application ने तो थेट वापरला:

```
container.innerHTML = userInput;
```

तर काही परिस्थितींमध्ये injected event handler execute होऊ शकतो.

हे XSS चे उदाहरण आहे.

Production solution:

जर user input फक्त text म्हणून दाखवायचा असेल:

```
container.textContent = userInput;
```

जर application ला user-authored HTML support करायचे असेल, तर योग्य HTML sanitization आणि security controls आवश्यक आहेत. फक्त input escaping किंवा CSP वर सर्व परिस्थितींमध्ये अवलंबून राहू नये.

Experienced developer विचारतो: हा data trusted आहे का? तो HTML म्हणून parse करायचा आहे की plain text म्हणून दाखवायचा आहे?

Scenario 12: `<img>` चे `alt` attribute नसेल तर?

```
<img src="student.jpg">
```

Image दिसू शकते. पण accessibility साठी ती अपुरी असू शकते.

Content image असल्यास:

```
<img src="student.jpg" alt="Student working on a laptop">
```

Decorative image असल्यास:

```
<img src="decoration.svg" alt="">
```

Industry insight: `alt` text हा फक्त SEO साठी नाही. तो screen-reader users साठी image चा अर्थ पोहोचवण्यास मदत करतो.

Scenario 13: `<button>` ऐवजी clickable `<div>` वापरल्यास?

```
<div onclick="submitForm()">Submit</div>
```

Mouse click वर हे काम करू शकते. पण native button सारखे keyboard behavior, semantics आणि accessibility आपोआप मिळत नाहीत.

योग्य पर्याय:

```
<button type="button" onclick="submitForm()">
    Submit
</button>
```

Production insight: शक्यतो native semantic HTML वापर. त्यामुळे accessibility, keyboard support आणि maintainability सुधारतात.

Scenario 14: Form मध्ये `type` न दिलेला button

```
<form>
    <input name="email">
    <button>Save</button>
</form>
```

Tricky point: Form च्या संदर्भात `<button>` चा default type सामान्यतः `submit` असतो.

म्हणून Save button click केल्यावर form submit होऊ शकतो.

जर button फक्त UI action साठी असेल:

```
<button type="button">Open Preview</button>
```

Submit करायचा असेल:

```
<button type="submit">Save</button>
```

Industry insight: अनावश्यक form submission, page reload आणि state loss टाळण्यासाठी button चा type स्पष्टपणे लिहिणे चांगली सवय आहे.

Level 4 — Browser Performance आणि Rendering

Scenario 15: `<head>` मध्ये मोठी JavaScript file ठेवली तर?

```
<head>
    <script src="/app.js"></script>
</head>
```

सामान्य external classic script मध्ये `async` किंवा `defer` नसल्यास browser script fetch आणि execute करण्यासाठी HTML parsing थांबवू शकतो.

त्यामुळे मोठ्या script मुळे initial rendering उशिरा होऊ शकते.

एक पर्याय:

```
<script src="/app.js" defer></script>
```

`defer` असलेली external classic script HTML parsing सोबत download होऊ शकते आणि document parsing पूर्ण झाल्यानंतर execute होते.

महत्त्वाचे: `async`, `defer`, module scripts आणि dynamically inserted scripts यांचे execution rules वेगवेगळे आहेत. प्रत्येक script साठी एकच universal rule लागू होत नाही.

Industry insight: Production मध्ये Core Web Vitals, script execution cost आणि critical rendering path विचारात घ्यावा.

Scenario 16: HTML मध्ये खूप nested elements असतील तर?

```
<div>
    <div>
        <div>
            <div>
                <p>Hello</p>
            </div>
        </div>
    </div>
</div>
```

हे HTML render होऊ शकते.

पण केवळ nesting जास्त आहे म्हणून application आपोआप slow होईल असे नाही.

Performance वर परिणाम अवलंबून असतो:

- DOM size

- CSS selectors
- Layout complexity
- JavaScript DOM operations
- Repeated style recalculation
- Rendering workload

Industry insight: अनावश्यक wrapper elements टाळ, पण performance समस्यांचे निदान अंदाजाने न करता profiling करून कर.

Level 5 — Advanced Debugging Exercises

खालील scenarios साठी आधी स्वतः उत्तर दे आणि मग solution पाहा.

1. `<p>One<div>Two</div>Three</p>` दिल्यास काय होते?

div हा p चाच child राहतो

div सुरू होताना p बंद होऊ शकतो; नंतरचा closing p stray markup म्हणून handle होऊ शकतो

Browser नेहमी syntax error देतो

2. `document.querySelector('.missing')` element सापडला नाही तर काय return करतो?

Empty object

null

undefined

3. `<img src='x.jpg'>` image load झाली नाही तर?

DOM element आपोआप delete होतो

Image element DOM मध्ये राहू शकतो, पण image resource render होण्यात failure होतो

संपूर्ण HTML parsing थांबते

4. `<script defer src='app.js'></script>` बद्दल योग्य विधान कोणते?

Script download होईपर्यंत HTML parsing नेहमी थांबते

External classic script defer असल्यास parsing चालू राहू शकते आणि execution parsing पूर्ण झाल्यानंतर होते

defer म्हणजे script कधीच execute होत नाही

0 of 4 answered

Try again

3+ Years Engineer साठी शेवटची मानसिक चौकट

एखादा HTML bug समोर आला की खालील क्रमाने विचार कर:

1. Source

HTML मध्ये प्रत्यक्षात काय लिहिले आहे? DOCTYPE, tags, attributes आणि nesting योग्य आहेत का?

2. Parser and DOM

Browser ने कोणता DOM तयार केला? Missing elements, implicit nodes किंवा malformed markup आहे का?

3. CSS and Rendering

Element DOM मध्ये आहे का? तो hidden आहे का? Layout, paint किंवा compositing मध्ये समस्या आहे का?

4. Production Quality

Accessibility, security, performance, semantics आणि browser compatibility तपास.

तुझ्यासाठी मुख्य सल्ला: HTML शिकताना प्रत्येक tag चे syntax आणि output एवढ्यावर थांबू नकोस. Browser ने त्याचे parsing कसे केले, DOM मध्ये काय तयार झाले आणि production मध्ये त्याचा काय परिणाम होईल, हे समजून घे. हाच basic frontend knowledge आणि मजबूत engineering understanding यांमधील फरक आहे.

Technical Addendum — Advanced HTML and Browser Engineering

A. Semantic HTML: meaning is part of the interface

English: Semantic elements describe the role of content, not merely its appearance. A `<nav>` identifies a navigation region, `<main>` identifies the primary page content, `<button>` represents an action, and `<a href="…">` represents navigation to a resource.

मराठी: Semantic HTML म्हणजे element फक्त दिसण्यासाठी न वापरता त्याचा अर्थ आणि भूमिका योग्य प्रकारे व्यक्त करणे. यामुळे browser, screen reader, search engine आणि maintainers यांना page समजणे सोपे होते.

```
<header>
  <h1>Student Portal</h1>
  <nav aria-label="Primary">
    <a href="/courses">Courses</a>
    <a href="/profile">Profile</a>
  </nav>
</header>
<main>
  <h2>My Courses</h2>
  <p>Track your progress here.</p>
</main>
<footer>© 2026 Student Portal</footer>
```

Engineering rule: `<div>` is a generic container, not inherently bad. Use it when no more meaningful element fits. Do not choose semantic tags only because of their default visual styling.

B. Attributes, boolean attributes and `data-*`

An **attribute** supplies configuration or metadata to an element. Some attributes are strings (`href` , `src` , `alt`); some are boolean (`disabled` , `required` , `checked`). For boolean attributes, presence generally means true, even if written as `disabled="false"` —that still counts as present. Remove the attribute to make it false.

```
<input required>
<button disabled>Saving…</button>
<div data-user-id="42" data-state="ready"></div>
```

```
const panel = document.querySelector('[data-state="ready"]');
console.log(panel.dataset.userId); // "42" (a string)
```

मराठी: Boolean attribute साठी `disabled="false"` लिहिल्याने button enable होत नाही. Attribute अस्तित्वात असेल तर तो disabled राहतो. `data-*` custom application data साठी आहे; त्यात secret किंवा authorization decision ठेवू नये.

C. Forms: markup, browser behavior and backend contract

```
<form action="/search" method="get">
  <label for="q">Search products</label>
  <input id="q" name="q" type="search" required>
  <button type="submit">Search</button>
  <button type="reset">Reset</button>
  <button type="button" id="helpButton">Help</button>
</form>
```

- `id` connects the label using `for`; `name` identifies a field in submitted form data.
- `GET` commonly places submitted fields in the URL query string. Do not put secrets in query parameters.
- `POST` sends form data in the request body, but **POST alone does not provide encryption**; HTTPS is needed in transit.
- Native constraints such as `required`, `min`, `max`, `pattern` and input types improve UX. The backend must validate independently because client-side validation can be bypassed.
- A button inside a form defaults to submit behavior when no type is specified. Be explicit.
- A label improves usability and accessible naming; placeholder text is not a replacement for a label.

D. Accessibility tree and keyboard behavior

The DOM tree is not the same thing as the **accessibility tree**. Browsers expose semantic information to assistive technologies through accessibility APIs. CSS visibility, ARIA attributes, roles, names, states and native semantics influence what assistive technology receives.

Prefer native elements:

```
<button type="button" id="save">Save changes</button>
<a href="/settings">Open settings</a>
```

Avoid using a clickable `<div>` as a substitute for a button unless you also deliberately implement keyboard operation, focusability, role, state and expected interaction behavior. Native `<button>` already supports keyboard activation, focus behavior and platform semantics.

ARIA rule of thumb: First use correct native HTML. Add ARIA only when native semantics do not express the required meaning or state. Incorrect ARIA can make accessibility worse.

E. Script loading and HTML parser interaction

Parser-blocking classic script

```
<script src="app.js"></script>
```

For a classic external script encountered by the parser, the browser commonly pauses parsing while it fetches and executes the script, unless other mechanisms change the loading behavior. This is why placing scripts carelessly in the head can delay parsing.

`defer`

```
<script defer src="app.js"></script>
```

For an external classic script, downloading can occur while HTML parsing continues; execution occurs after parsing finishes and before `DOMContentLoaded`, preserving order among deferred classic scripts.

`async`

```
<script async src="analytics.js"></script>
```

An async script executes as soon as it is ready, so its execution order relative to other async scripts is not guaranteed. Use it only when that order does not matter.

Module scripts

```
<script type="module" src="main.js"></script>
```

Module scripts are deferred by default, support imports/exports and run in strict mode. Cross-origin, caching and dependency behavior still need to be understood for production applications.

Important: `defer` and `async` do not mean the same thing. Select the loading strategy based on dependencies and execution-order requirements, not as a memorized rule.

F. From HTML bytes to pixels: a practical mental model

1. **Response bytes arrive** over the network or local file system.
2. **Encoding is determined** and bytes are decoded to characters.
3. **HTML parser** tokenizes markup and builds the DOM; it applies HTML-specific error recovery.
4. **Resource discovery** finds stylesheets, scripts, images, fonts and other referenced resources. Browsers may use a preload scanner to discover some resources early.
5. **CSS parsing** builds CSSOM/style data. CSS and JavaScript can affect when rendering or parsing proceeds.
6. **Style calculation** determines applicable computed styles.
7. **Layout** calculates geometry for boxes that participate in layout.
8. **Paint** records visual drawing operations.

9. **Rasterization and compositing** turn drawing operations into pixels/layers and combine them for display.

This is a teaching model, not a guarantee that every frame performs every stage from scratch. A DOM update can trigger different amounts of work. A transform or opacity change may sometimes be composited without a full layout, while changing width can require layout. The browser decides based on the properties, dependencies and implementation.

Terms that should not be confused

- **DOM:** structured document nodes and APIs.
- **CSSOM:** parsed CSS rules and related style representation.
- **Layout/reflow:** geometry calculation.
- **Paint:** recording visual operations.
- **Rasterization:** converting drawing commands into pixels.
- **Compositing:** combining layers/surfaces into the final frame.
- **V8:** Chrome's JavaScript engine; it is not the HTML rendering engine.
- **Blink:** Chromium's rendering engine; it handles HTML/CSS parsing and rendering-related work.

G. Event loop, main thread and long tasks

JavaScript execution, event handling and parts of rendering coordination compete for time on the main thread. A long synchronous task can delay clicks, typing, timers and rendering opportunities.

```
 const button = document.querySelector('#load');
button.addEventListener('click', () => {
  // Avoid expensive synchronous loops on the main thread.
  console.log('Clicked');
});
```

मराठी: एखादा JavaScript task खूप वेळ चालला तर browser ला मध्येच user click process करणे किंवा पुढचा frame दाखवणे शक्य नसेल. अशा वेळी page “hang” झाल्यासारखा वाटतो. DevTools Performance panel वापरून long tasks आणि layout work तपासावे.

Do not assume that “JavaScript is single-threaded” means the entire browser is single-threaded. Browsers use multiple processes and threads; the exact architecture is implementation-dependent.

H. Resource loading, responsive images and lazy loading

```
<img
  src="student-small.jpg"
  srcset="student-small.jpg 480w, student-large.jpg 1200w"
  sizes="(max-width: 600px) 100vw, 600px"
  alt="Student working on a laptop"
  width="1200"
  height="800"
  loading="lazy">
```

- `srcset` and `sizes` help the browser choose an appropriate image resource.
- Explicit `width` and `height` allow the browser to reserve space and reduce layout shifts.

- `loading="lazy"` can defer off-screen image loading. Avoid lazy-loading the primary above-the-fold/LCP image without reason.
- `alt` should convey the image's purpose in context. Decorative images generally use `alt=""`; meaningful images need useful alternatives.
- Use caching and compression at the delivery layer too; HTML alone does not make a page fast.

I. Security: safe DOM manipulation and XSS

Unsafe pattern:

```
// Dangerous if userInput is untrusted and contains markup/script payloads.
container.innerHTML = userInput;
```

Safer for plain text:

```
container.textContent = userInput;
```

If the product intentionally accepts user-authored HTML, use a well-maintained sanitizer with an explicit allowlist and review the exact insertion context. Avoid constructing JavaScript, CSS, HTML or URLs by concatenating untrusted strings. Validate URLs and allow only expected schemes/hosts where appropriate. Content Security Policy (CSP) is useful defense in depth, not a substitute for safe coding.

मराठी: Browser मध्ये दिसणारा text हा नेहमी harmless असतो असे नाही. User input HTML म्हणून parse केला गेला, तर attacker ने दिलेला markup किंवा executable behavior चालण्याचा धोका असू शकतो. Input कुठे वापरतोस—text, HTML attribute, URL, CSS की JavaScript—यावर योग्य protection अवलंबून असते.

J. Performance: measure before optimizing

Practical checks:

- Lighthouse / Performance panel मध्ये loading आणि interaction bottlenecks शोध.
- Network panel मध्ये request count, cache headers, transferred bytes, slow resources तपास.
- Elements panel मध्ये unexpected DOM nodes आणि CSS visibility तपास.
- Layout shifts, large images, render-blocking resources आणि long main-thread tasks शोध.
- DOM size कमी करणे उपयोगी असू शकते, पण फक्त nesting जास्त आहे म्हणून page नक्की slow होतो असा निष्कर्ष काढू नको.
- CSS selectors किंवा JavaScript patterns बद्दल blanket claims करण्याऐवजी real trace आणि measurements वर निर्णय घे.

K. Debugging checklist for a production issue

1. **Reproduce:** exact browser, URL, viewport, user steps and console error नोंदव.
2. **Inspect response:** Network panel मध्ये status, response headers, MIME type, redirects आणि HTML response तपास.
3. **Inspect source vs DOM:** View Source आणि Elements panel compare कर.
4. **Check semantics/structure:** malformed nesting, duplicate IDs, missing labels, wrong element choice आणि implicit nodes तपास.

5. **Check CSS:** computed styles, box model, `display`, `visibility`, `opacity`, stacking context and overflow तपास.
6. **Check script timing:** Console errors, async/defer/module behavior आणि DOM readiness तपास.
7. **Check resources:** image/font/CSS/JS requests fail होत आहेत का ते पाहा.
8. **Check accessibility:** keyboard-only flow, focus order, accessible name, contrast and screen-reader semantics.
9. **Check security:** unsafe HTML insertion, untrusted URLs, form validation and server-side validation.
10. **Measure:** Performance trace घेऊन actual bottleneck शोध.

L. Additional interview questions with model answers

1. Does the browser display the HTML source directly?

No. The browser decodes the response, parses HTML and constructs a DOM; CSS and other resources are processed and rendering work produces pixels. The DOM can differ from the literal source because of parsing rules and later JavaScript changes.

2. Why can an element exist in DevTools but not be visible?

It may have `display: none`, `visibility: hidden`, zero size, clipping, off-screen positioning, opacity, overlap/stacking issues, or no visible content. The element may also be outside the viewport or hidden by an ancestor. DOM presence alone does not guarantee visible pixels.

3. Is `DOMContentLoaded` the same as page load?

No. `DOMContentLoaded` fires when the document has been parsed and deferred/module scripts have executed (with details depending on module dependencies). The `load` event waits for the page's dependent resources such as images to finish loading, subject to browser rules. Neither event means every later lazy resource is loaded.

4. Does `alt` text always need to describe the image literally?

It should serve the image's purpose in context. For a functional image, describe the action/destination; for informative images, convey relevant information; for decorative images, use empty alt text so screen readers can skip it.

5. Does client-side validation protect a backend?

No. Client-side validation improves usability but can be bypassed. The server must validate, authorize and safely process submitted data independently.

6. Is the DOM the same as the accessibility tree?

No. The accessibility tree is a platform-facing representation derived from DOM semantics, computed state and accessibility mappings. It may omit nodes that are not exposed and may represent roles, names and states differently.

7. Why should a developer prefer `<button>` over a clickable `<div>`?

A native button already has keyboard interaction, focusability, semantics and accessibility mappings. A clickable div requires these behaviors to be implemented correctly and consistently.

8. What is the difference between layout and paint?

Layout computes geometry; paint records how elements should be drawn. Some changes trigger both; other changes may avoid layout. Performance decisions should be confirmed with browser profiling.

M. Hands-on lab: verify instead of memorizing

Create a folder and save each example as an `.html` file. Open it in a browser and use DevTools.

Lab 1 — Parser-created structure

```
<!doctype html>
<title>Implicit structure</title>
<p>First paragraph
<p>Second paragraph
```

Questions: What does View Source show? What does Elements show? How does the parser handle the second `<p>`? Do not assume the source is rewritten.

Lab 2 — Table body

```
<table>
  <tr><td>Java</td></tr>
</table>
```

Inspect whether the DOM includes a `<tbody>` node. Explain why parser-created structure can differ from source text.

Lab 3 — Boolean attribute

```
<button disabled="false">Still disabled</button>
```

Explain why the button remains disabled and test removing the attribute.

Lab 4 — Form defaults

```
<form>
  <button>Unexpected submit?</button>
  <button type="button">No submit</button>
</form>
```

Add a submit listener and observe which button triggers it.

Lab 5 — Text versus HTML insertion

```
<div id="out"></div>
<script>
  const out = document.querySelector('#out');
  out.textContent = '<strong>plain text</strong>';
  // Compare with out.innerHTML = '<strong>parsed markup</strong>'
</script>
```

Observe the DOM and rendered result. Use only safe, controlled test strings.

Lab 6 — Deferred versus async order

Create two scripts that log messages and take different amounts of time to download. Compare classic scripts, `defer`, `async`, and `type="module"`. Network timing varies, so repeat tests rather than expecting one fixed order for async scripts.

N. Reliable documentation for continued study

Use these sources for current behavior and precise definitions:

- MDN HTML overview: https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN HTML elements reference: https://developer.mozilla.org/en-US/docs/Web/HTML/Element
- MDN DOM introduction: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- WHATWG HTML Living Standard: https://html.spec.whatwg.org/
- MDN script element: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/script
- MDN forms guide: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms
- web.dev rendering performance: https://web.dev/articles/rendering-performance
- OWASP Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Final mental model

Source → bytes/encoding → HTML parser → DOM + resource discovery → CSS/style calculation → layout → paint → rasterization/compositing → pixels. JavaScript can observe and mutate the DOM, load more resources and trigger new rendering work. Production engineering adds semantics, accessibility, security, performance measurement and compatibility.

मराठीत लक्षात ठेव: **फक्त tag पाठ करू नकोस. Browser ने source कसा parse केला, DOM मध्ये काय तयार झाले, CSS ने काय बदलले, script कधी execute झाले आणि user ला दिसणाऱ्या pixels वर काय परिणाम झाला—हा पूर्ण flow समजून घे.**